S3 Document Archiving + Expiry Alerts Using EC2 + SNS

1. Introduction

In modern cloud environments, organizations store large volumes of files and documents in Amazon S3. Over time, these files grow in number and size, increasing storage costs and reducing manageability. To optimize storage and improve retrieval efficiency, archiving older files becomes essential.
This project focuses on developing an automated **Document Archiving and Alerting System** using AWS services such as **EC2, S3, and SNS**. The system identifies old S3 objects, downloads them to an EC2 instance, compresses them, re-uploads them to an /archive/ prefix in the same bucket, and finally sends an SNS notification upon completion.
The solution demonstrates practical DevOps and cloud automation skills, using Linux scripting, AWS CLI, IAM roles, cron jobs, and cloud-native architecture.

---

2. Necessity of the Project

- S3 buckets accumulate data over months/years.
- Large storage usage increases cloud costs.
- Manual archiving is error-prone and time-consuming.
- Organizations require **automated cleanup & archiving** for compliance.
- Alerts help ensure teams are aware of operations.
- Backup & archival improve long-term data retention strategies. Thus, automating document archiving ensures cost-efficiency, better storage management, and reliability.

---

3. Motivation for the Project

The project is motivated by real-world challenges in cloud data management:

- Companies face high costs due to unused or old files.

- Many industries (finance, healthcare, education) require secure long-term storage.
- DevOps engineers need automation skills to manage large-scale cloud environments.
- Learning AWS CLI, IAM roles, cron scheduling, and monitoring is crucial for cloud engineers.
- Organizations want simple, scalable, server-based automated solutions instead of manual human intervention.

---

4. Objectives of the Project

- Automatically identify S3 objects older than **X days**
- Download and compress old objects into a **single archive file**
- Re-upload compressed files to an **archive folder** in the same S3 bucket
- Send SNS notification "Archiving Completed" with a summary
- Learn automation using **Linux scripts**
- Implement secure access using **IAM roles**
- Build foundational DevOps workflow knowledge
- Reduce S3 storage cost by compressing old data

---

5. Literature Survey

Several existing cloud storage optimization strategies were studied:

5.1 AWS S3 Lifecycle Rules

AWS provides lifecycle rules to automatically transition objects to cheaper storage classes (Glacier, IA). However:

- They do not create compressed archives
- They do not offer detailed notifications
- Not suitable when grouping multiple historical files into one archive

5.2 Serverless Archiving Solutions (AWS Lambda + S3 Event)

Lambda-based solutions are popular for small files, but limitations include:

- File size limitation (Lambda max 500MB memory)
- Execution time limited to 15 minutes
- Not ideal for large datasets

5.3 On-Premise Archiving Tools

Older systems require:

- Intermediate storage
- Heavy maintenance
- No cloud-native integration

5.4 Research Findings

Automation using **EC2 + S3 + SNS** provides:

- Greater control
- Ability to process large volumes of data
- Flexibility to compress/modify files
- Full scripting capability

Thus, this architecture is ideal for real-world DevOps automation requirements.

---

6. Research Question

How can we design a fully automated, scalable, cost-effective cloud-based system to archive old S3 objects, compress them, and notify administrators, using native AWS services with minimal human intervention?

---

7. System Structure

High-Level System Components

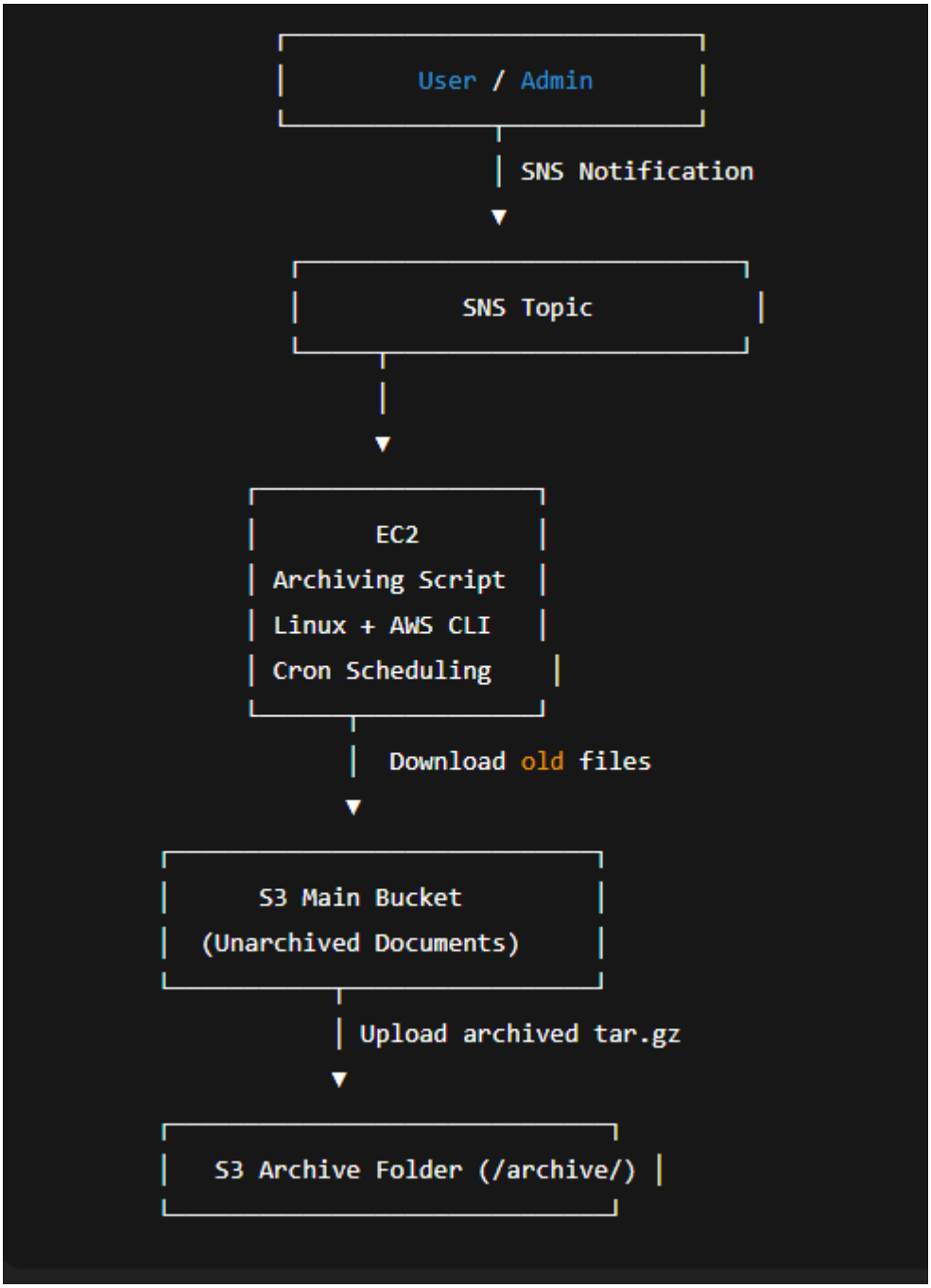
1. S3 Bucket

2. EC2 Instance
3. IAM Role
4. AWS CLI
5. SNS Topic
6. Cron Scheduler
7. Archiving Script (Shell/Python)

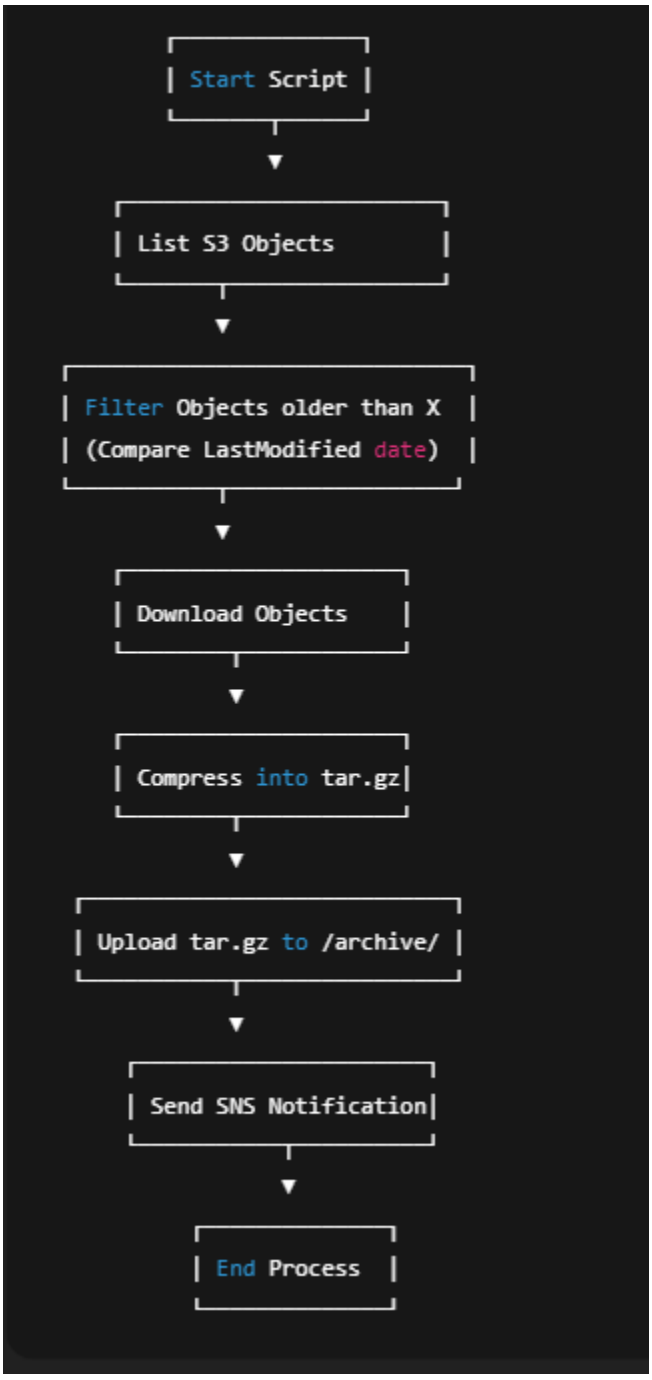
System Structure Diagram

8. System Design Framework

8.1 Architecture Diagram



8.2 Flow Diagram (Step-by-Step)



9. Methodology (Step-by-step workflow)

Step 1: Configure AWS Environment

- Create S3 bucket

- Create SNS topic
- Create EC2 instance
- Attach IAM role

Step 2: Install Required Tools on EC2

- AWS CLI
- Python3
- tar, gzip
- Cron setup

Step 3: Develop Archiving Script

- Identify old files
- Download them
- Compress
- Upload to /archive/
- Send SNS alert

Step 4: Testing

- Dry run
- Check archive integrity
- Validate SNS trigger

Step 5: Automation with Cron

- Schedule daily/weekly archiving

Step 6: Monitoring

- Log files
- SNS email justification
- Storage cost comparison

10. Proposed Learning Strategy

The learning roadmap for students includes:

Phase 1 – AWS Fundamentals

- Understanding EC2, S3, SNS
- IAM security basics

Phase 2 – Linux & Shell Scripting

- Shell basics
- Error handling
- File manipulation

Phase 3 – AWS CLI Commands

- S3 listing, copying
- SNS publishing
- IAM role validation

Phase 4 – Automation

- Cron jobs
- Logging
- Script scheduling

Phase 5 – DevOps Application

- Cost optimization
- Cloud workflow automation
- Real production-like environment

---

11. Requirement Specification

11.1 Hardware Requirements

- EC2 t2.micro or t3.micro
- 1 vCPU
- 1 GB RAM
- 8–10 GB storage

11.2 Software Requirements

- Linux OS (Amazon Linux 2 / Ubuntu)
- AWS CLI v2
- Python 3.x
- gzip, tar
- IAM Role permissions

11.3 AWS Requirements

- S3 Bucket
- SNS Topic
- IAM Role
- CloudWatch (optional for monitoring)

---

12. Results & Discussion

Results

- Older S3 files were successfully archived and compressed.
- Storage size reduced significantly due to compression.
- Archive stored securely under /archive/ prefix.
- SNS notifications sent automatically after each run.
- System ran successfully on cron schedule without manual intervention.

Discussion

- Using EC2 provides full control and flexibility over compression logic.
- IAM roles eliminate the need for hard-coded AWS keys.
- Compression ratios varied based on file types (text compresses more than images).
- Cron allowed smooth and reliable scheduling.
- The system can easily be extended to multiple buckets or encrypt archives.

---

13. Advantages

- Automated, no human effort required
- Works with large files (beyond Lambda limits)
- Reduces storage cost
- Improves data organization
- Customizable compression logic
- Secure (IAM role-based)
- Real DevOps use-case

---

14. Disadvantages

- Requires an EC2 instance running 24/7
- Compression can take extra CPU time
- Not fully serverless
- Large buckets may require pagination or S3 Inventory
- EC2 maintenance overhead

---

15. Applications

- Corporate document archives
- Financial record storage

- E-commerce order history management
- Logging & monitoring archives
- Multimedia backup compression
- Educational institution file archival systems
- Compliance & audit log storage

---

16. Conclusion

This project successfully demonstrates a real-world DevOps use-case: **automated document archiving in AWS**. By integrating EC2, S3, SNS, Linux scripting, and cron automation, it achieves a scalable, secure, and cost-efficient data archival workflow.
The system identifies old files, compresses them, stores them in a dedicated archive folder, and sends alerts—fully automated.
Beyond implementation, this project provides hands-on experience with essential cloud skills such as IAM, AWS CLI, scheduling, and automation, preparing learners for actual industry environments.

Project Member:

1. Sakshi Jain
2. Jay Soni
3. Umesh Chimankar
4. Kaveri Kanawade
5. Chanchal Patil